



INSTITUTO FEDERAL

Ceará

Campus Maracanaú

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
DO CEARÁ - CAMPUS MARACANAÚ
CONDENAÇÃO DO CURSO SUPERIOR DE TECNOLOGIA EM
ENGENHARIA DE CONTROLE E AUTOMAÇÃO**

LUAN ITALO MOTA SOUSA

Controladores Digitais

Planta Sallen-Key-Duino — Sistema de 2ª ordem

MARACANAÚ -CE
2024

1. RESUMO

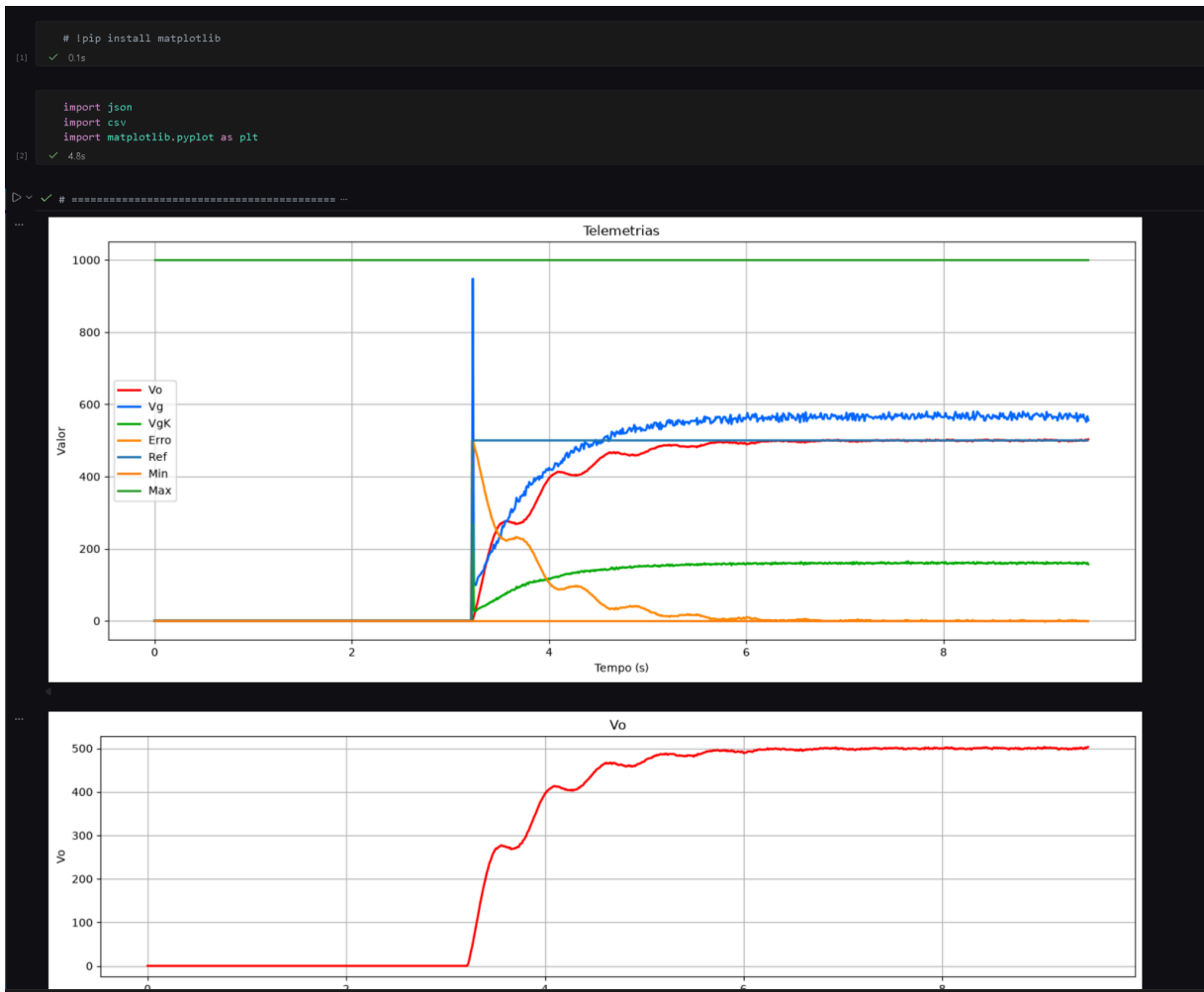
Este relatório documenta o projeto, a simulação e a implementação de controladores digitais aplicados à planta Sallen-Key-Duino, um sistema de segunda ordem. Partindo da modelagem da função de transferência contínua equivalente e de sua discretização por segurador de ordem zero (ZOH), foram projetados cinco controladores: Proporcional (P), Compensador de Avanço de fase, Proporcional-Integral (PI), Avanço+PI em cascata e Proporcional-Integral-Derivativo (PID). Para cada um avaliou-se a resposta ao degrau, o lugar das raízes e a resposta em frequência (Bode, Nyquist e Nichols). O objetivo do controle é rastrear a referência com estabilidade garantida. Os controladores com ação integral (PI, Avanço+PI e PID) eliminam o erro de regime, sendo o PID o de melhor desempenho global. Todos os projetos foram embarcados em um Arduino, com seleção do controlador via comando serial, permitindo comparar os resultados teóricos com os medidos na placa.

2. Modelagem da Planta

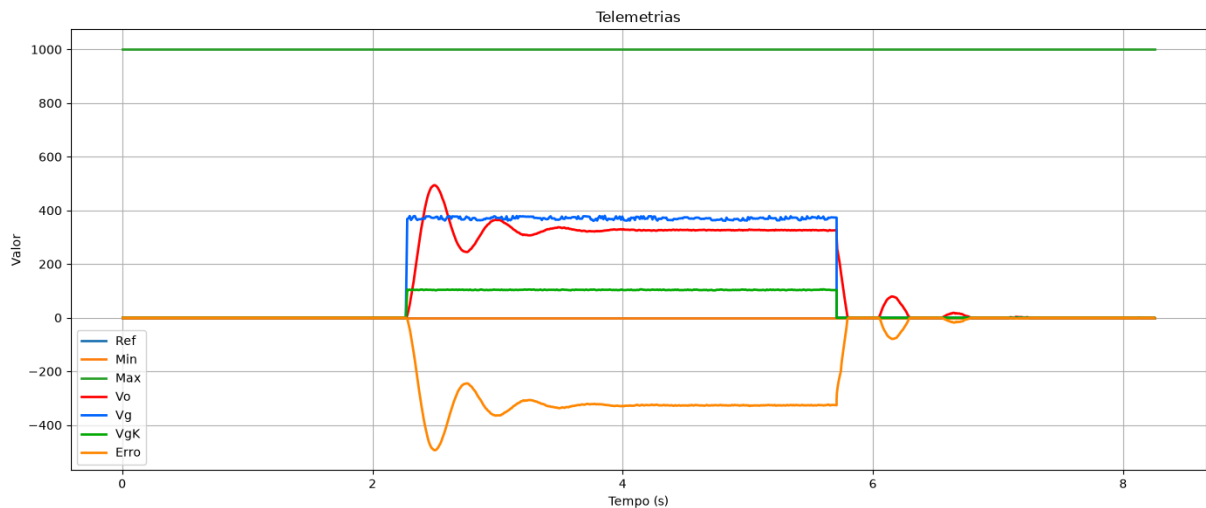
O processo de obtenção da função de transferência da planta se deu da seguinte forma: primeiro, capturou-se a forma de onda do Arduino. Para isso, foi utilizada uma extensão no VS Code que permite exportar os gráficos no formato JSON.



Após a exportação em JSON, foi desenvolvido um código em Python(gráfico.ipynb-git) para converter o formato JSON em CSV, possibilitando assim sua utilização no MATLAB.



Esse foi o gráfico que obtivemos após esses passos. o gráfico abaixo mostra a resposta da planta sem controle, com uma entrada degrau de 500.



A segunda parte foi a obtenção da caracterização da planta. Como já sabemos que se trata de um Sallen-Key, ela deve ter como base a seguinte função:

$$G(s) := \frac{\omega^2 \cdot k}{s^2 + (2 \cdot \zeta \cdot \omega) s + \omega^2}$$

A primeira a se calculado e o K que é o valor estável dividido pela o valor que deveria ter neste caso seria:

$$k := \frac{326}{500} = 0.652 \blacksquare$$

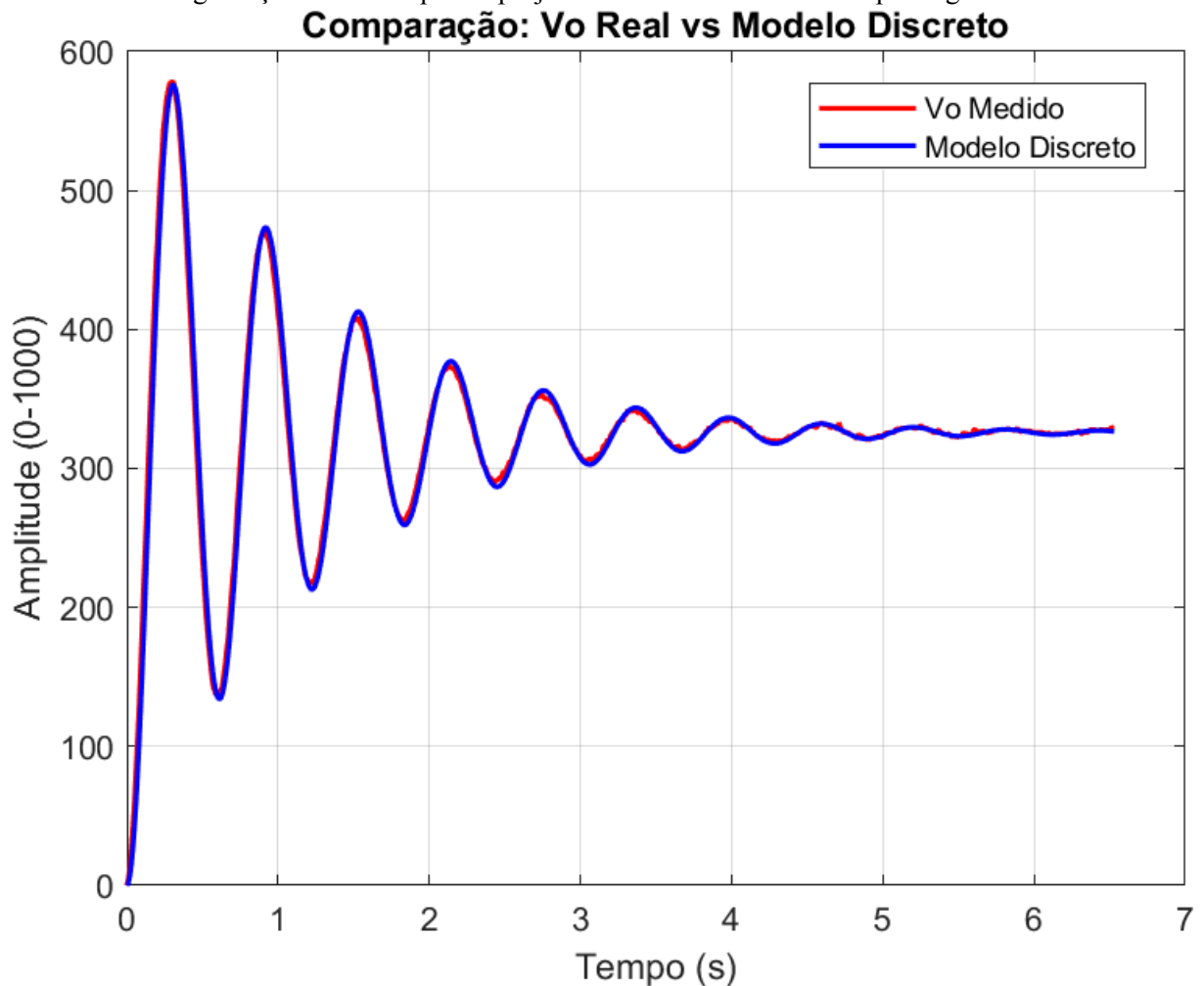
O segundo foi o ω_n , que é a frequência natural do sistema, dado por 2π dividido pelo período. Esse período é obtido medindo-se a distância entre dois picos consecutivos, que resultou em 61 amostras. Sabendo que T é 10 ms, é possível calcular o ω_n .

$$\omega := \frac{2\pi}{n \cdot t} = 10.3$$

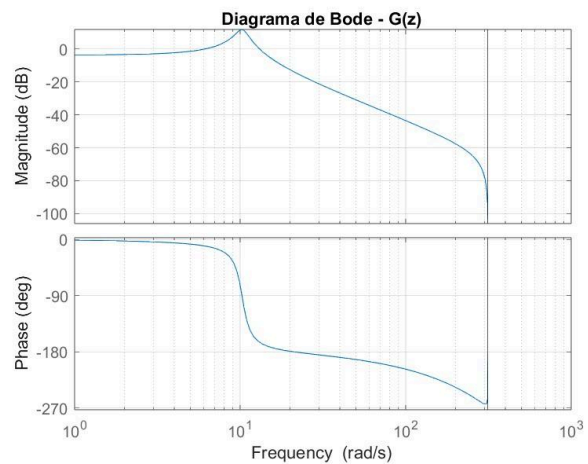
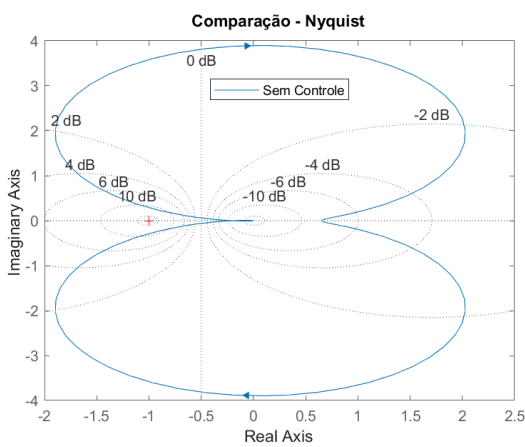
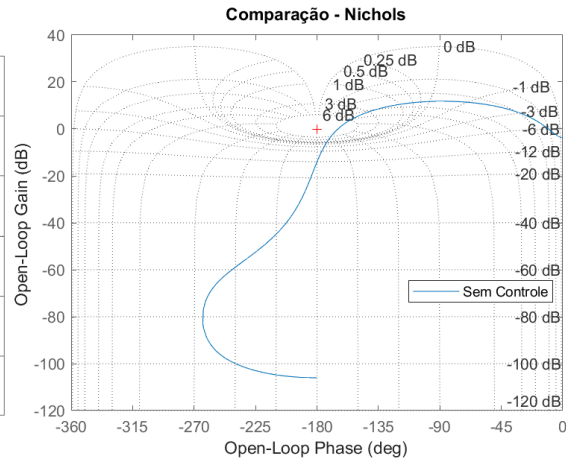
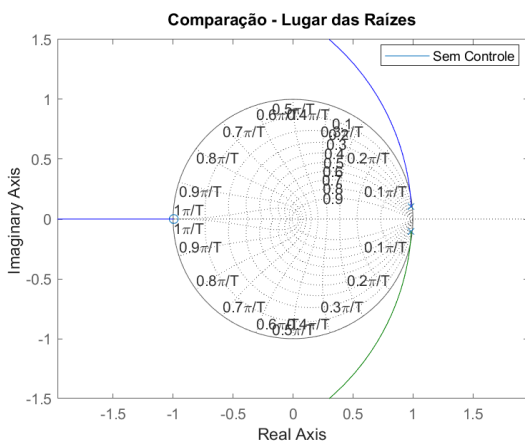
Por fim, determinou-se o coeficiente de amortecimento (ξ) por meio das fórmulas apresentadas a seguir.

$$\xi := \frac{-\ln(M_p)}{\sqrt{\pi^2 + \ln(M_p)^2}} = 0.084 \blacksquare \quad M_p := \frac{578 - 327}{327} = 0.768 \blacksquare$$

Com a planta agora caracterizada, é possível comparar o valor obtido do Arduino com a resposta da função de transferência levantada. O gráfico abaixo mostra que a curva real e a da caracterização ficaram bastante próximas, o que valida o modelo adotado. Essa boa correspondência entre o sinal medido e o modelo teórico confirma que os parâmetros estimados (k , ξ e ω_n) representam adequadamente o comportamento da planta. Dessa forma, a função de transferência obtida pode ser utilizada com segurança como base para o projeto dos controladores nas etapas seguintes.



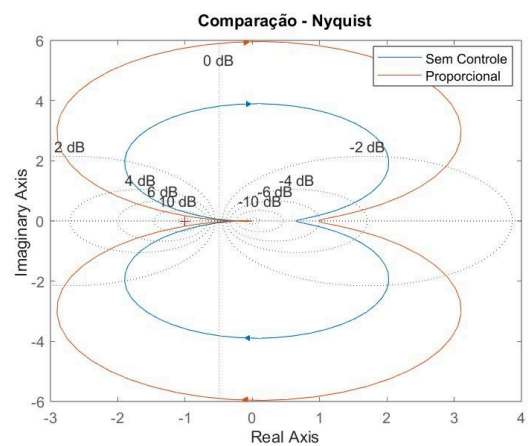
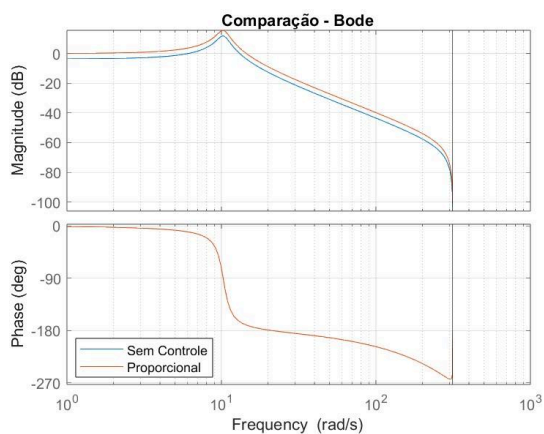
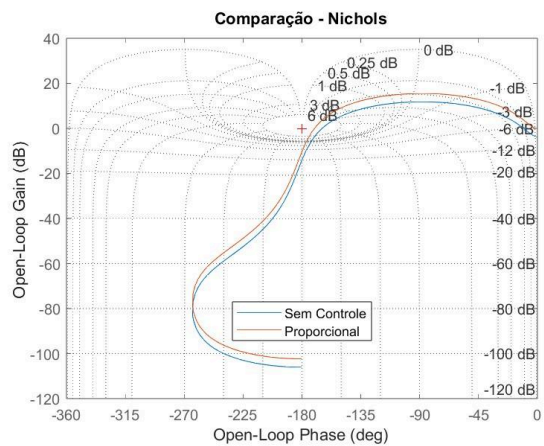
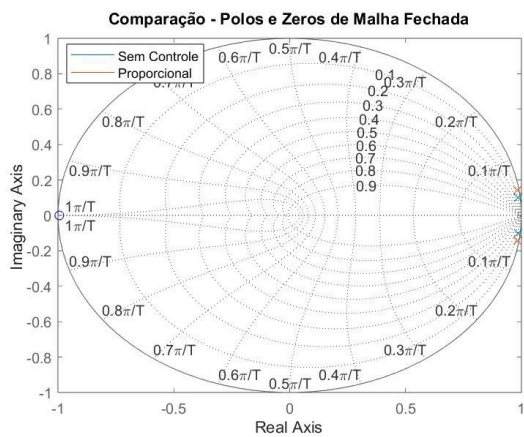
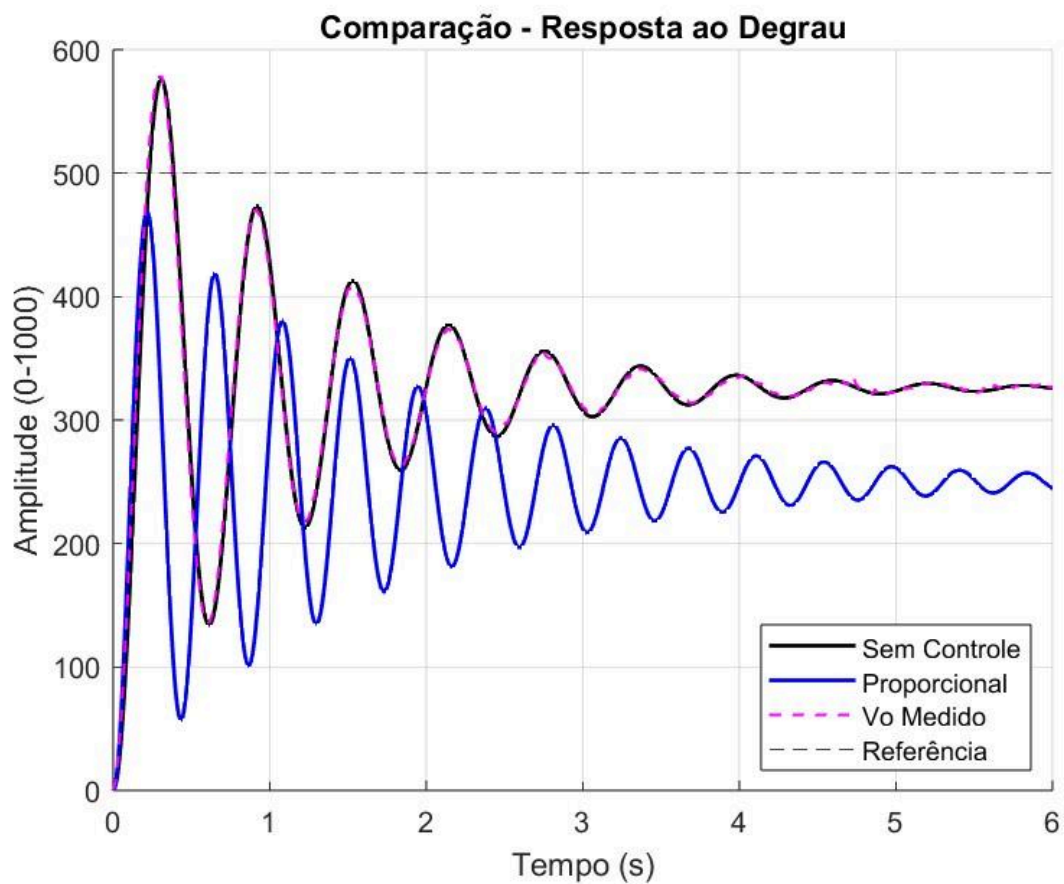
A imagem abaixo mostra o lugar das raízes do sistema sem controle e suas análises de Nichols, nyquist e bode.



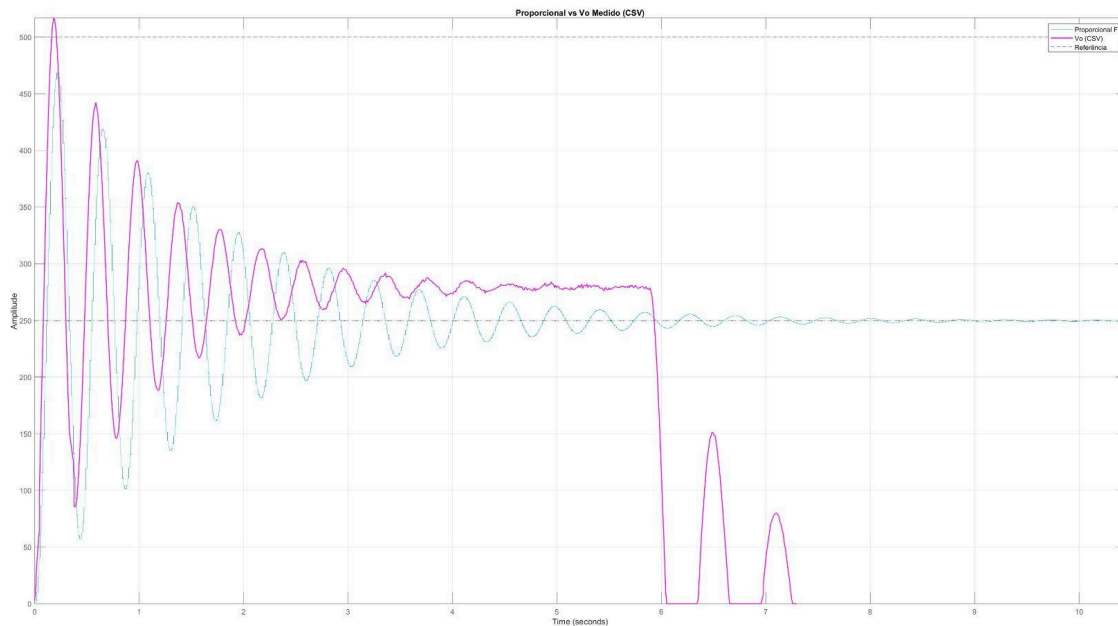
O que se pode concluir com isso é que o sistema sem controle apresenta um sobressinal de 76.7%, erro de regime de 34.8% e tempo de estabilização de aproximadamente 4 s, além de pouca margem para que apenas a modificação do ganho melhore a resposta do sistema.

3. Controlador Proporcional.

Para o controlador proporcional, como já dito anteriormente, não há muita margem para aumentar o ganho e obter valores aceitáveis de sobressinal e tempo de estabilização. Ainda assim, optou-se por um ganho de 1.53. Em malha aberta, ele zera o erro em regime permanente, porém em malha fechada, ele piora o resultado em todos os aspectos. Conclui-se, portanto, que para esse sistema apenas o ganho proporcional não é suficiente.

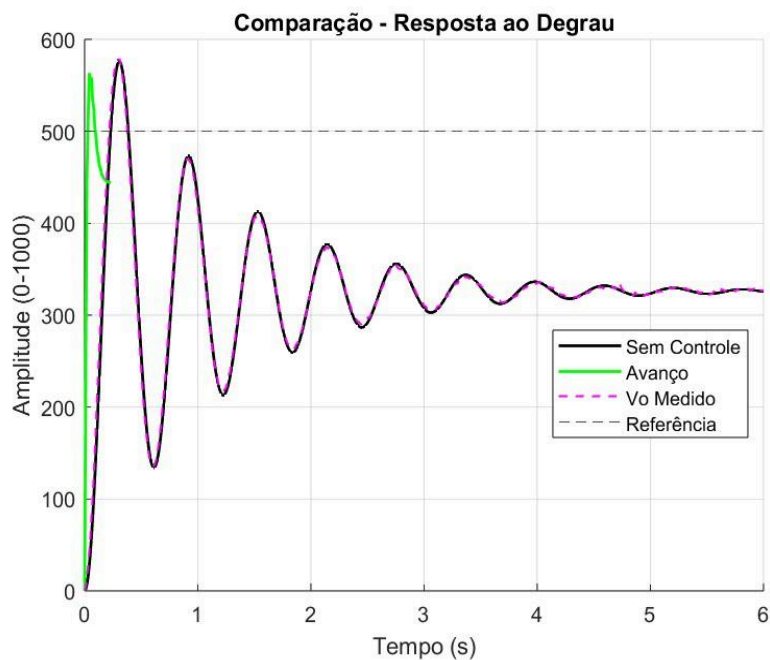


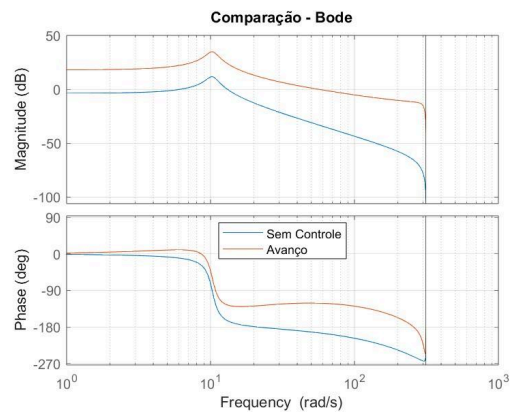
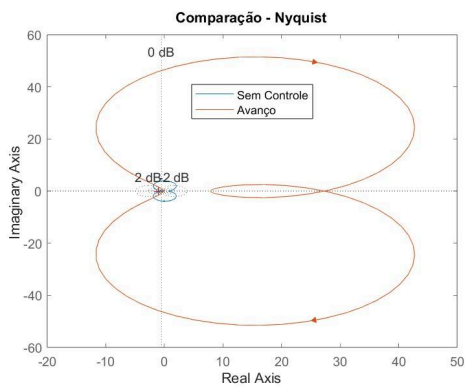
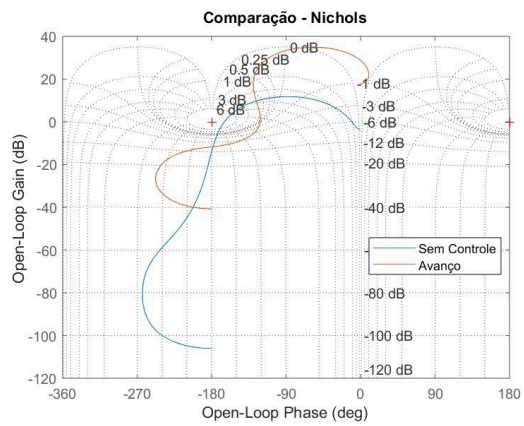
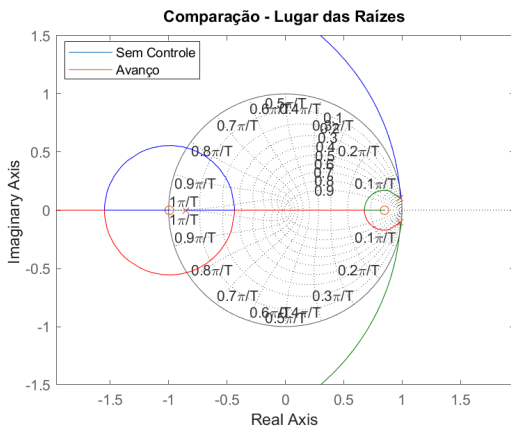
Na imagem abaixo temos a comparação entre o que foi planejado, em azul, e o que obtivemos no Arduino, em rosa. Perceba que, mesmo com a planta aparentemente bem modelada no MATLAB, há uma diferença considerável em relação ao real.



4. Avanço e Atraso

Em seguida, foi projetado o controlador de avanço e atraso. Como neste controlador temos um polo e um zero, que podem ser alocados onde acharmos necessário, além de um ganho, foi feita uma série de testes com várias configurações. A configuração escolhida foi o zero em 0.85 e o polo em -0.85, com um ganho de 150. O principal objetivo foi a velocidade de estabilização que ficou abaixo de 1s e com um sobressinal de 26.89% e um erro de 11.2%.





Na implementação do Arduino foi necessária a equação de diferenças deste controlador, obtida a partir da função de transferência $D(z) = K_c \cdot (z - z_c) / (z - p_c)$. Expandindo numerador e denominador, chega-se aos coeficientes:

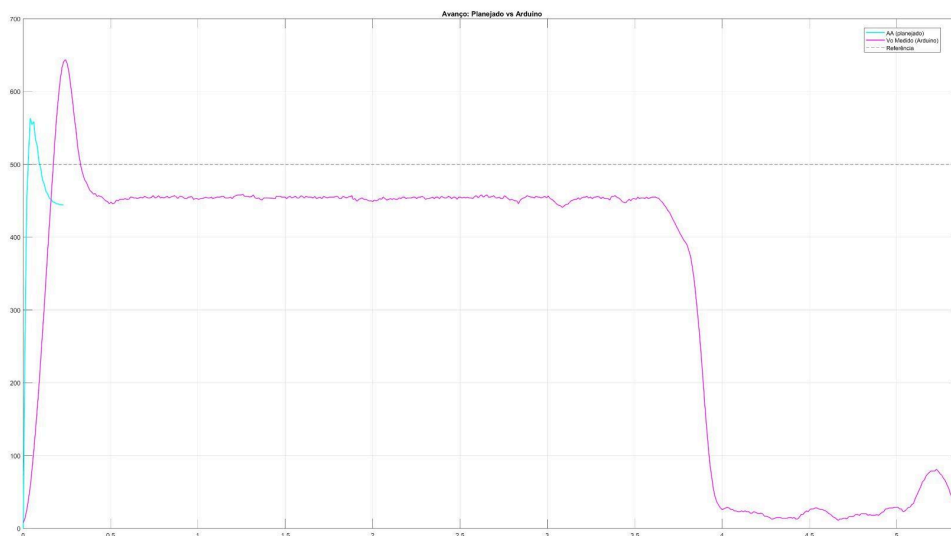
$$C(z) := K_c \cdot \frac{(z - z_c)}{(z - p_c)}$$

$$b_0 = K_c = 150$$

$$b_1 = -K_c \cdot z_c = -150 \times 0.85 = -127.5$$

$$a_1 = -p_c = -(-0.85) = 0.85$$

$$u[k] = b_0 \cdot e[k] + b_1 \cdot e[k-1] - a_1 \cdot u[k-1]$$

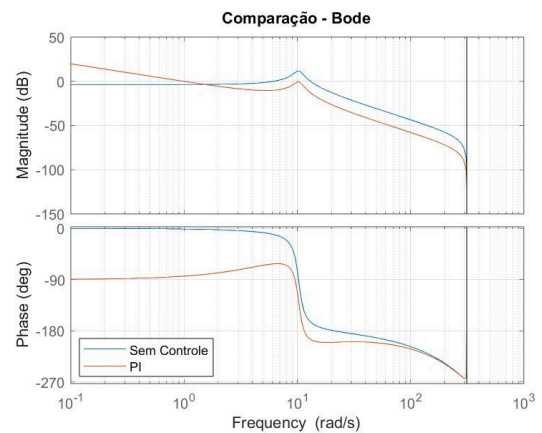
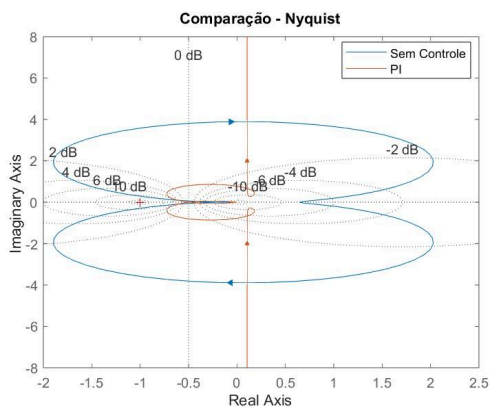
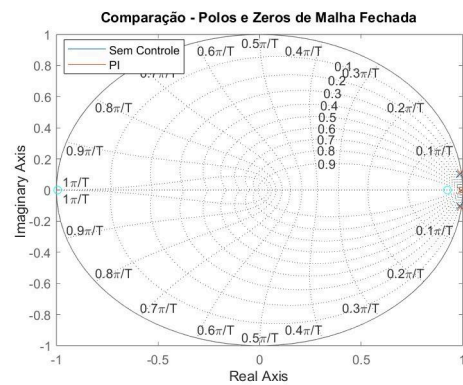
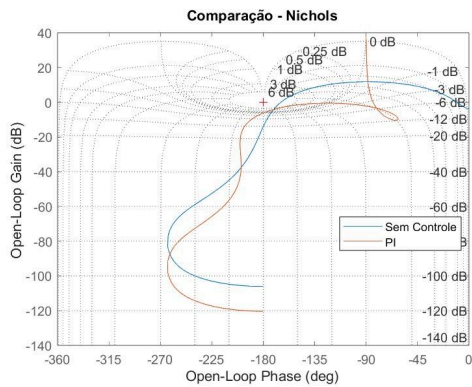
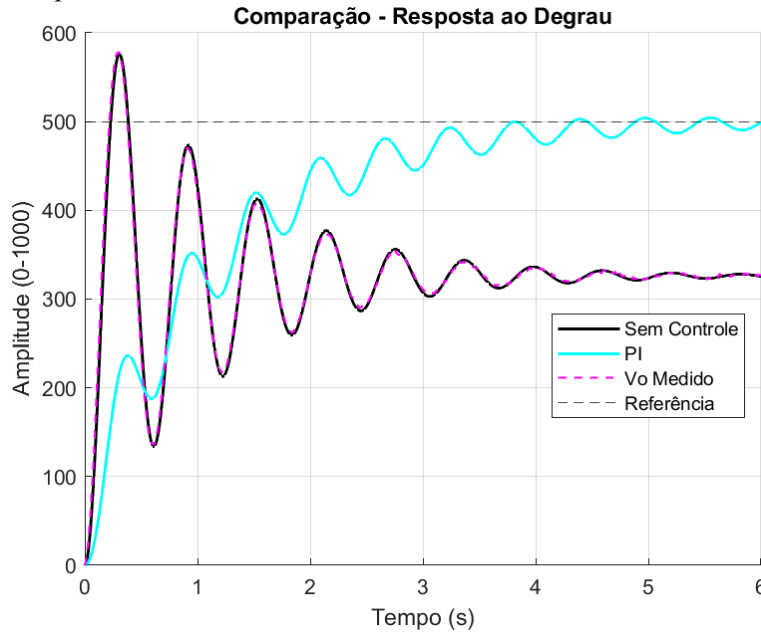


5. Controlador P I

Para o controlador PI, como ele tem a característica de erro zero em regime permanente para um sistema de 2ª ordem, o principal objetivo foi torná-lo o mais rápido possível e com pouco sobressinal.

$$C(z) := K_c \cdot \frac{(z - z_c)}{(Z - 1)}$$

Utilizando o modelo de ganho e zero, foram feitas várias configurações, e a escolhida foi $z_c = 0.925$ e $K_c = 0.2$. Assim, ele apresenta um sobressinal de menos de 1% e um tempo de estabilização de 5 s, porém ainda muito oscilatório.



Para a implementação no Arduino, foi necessário converter o modelo de ganho e zero para ganho proporcional e ganho integrativo:

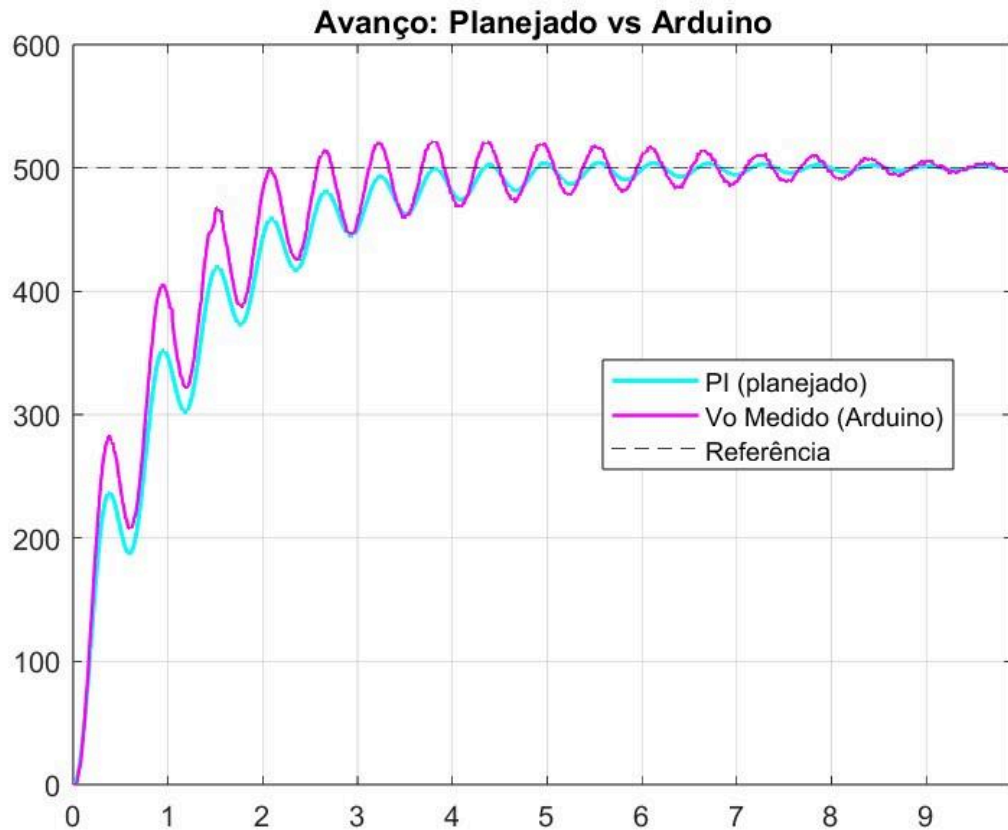
$$C(z) := K_c \cdot \frac{(z - z_c)}{(z - 1)} \quad C(z) := K_p \cdot \frac{(K_i \cdot T)}{(z - 1)}$$

zc_pi = 0.925; Kc_pi = 0.20;

Ki_PI = Kc_pi*(1 - zc_pi)/T;

Kp_pi = 0.2; Kc_pi = 1.5;

Assim como no controlador anterior, houve uma pequena diferença entre o obtido e o planejado, mas o sistema manteve as mesmas características.



6. Controlador Avanço + PI

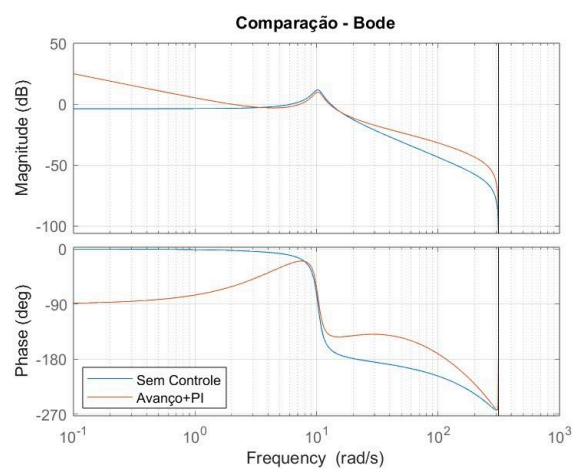
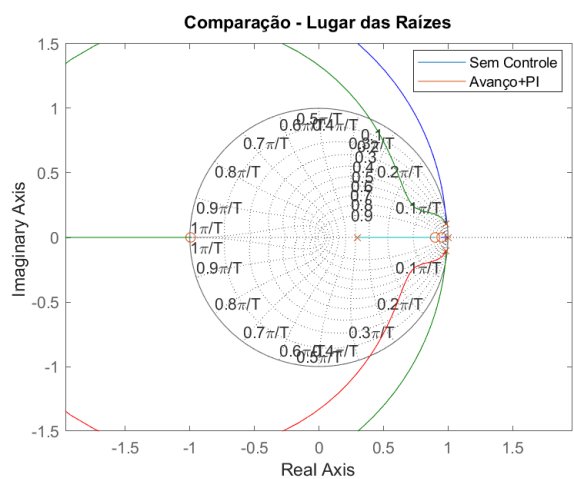
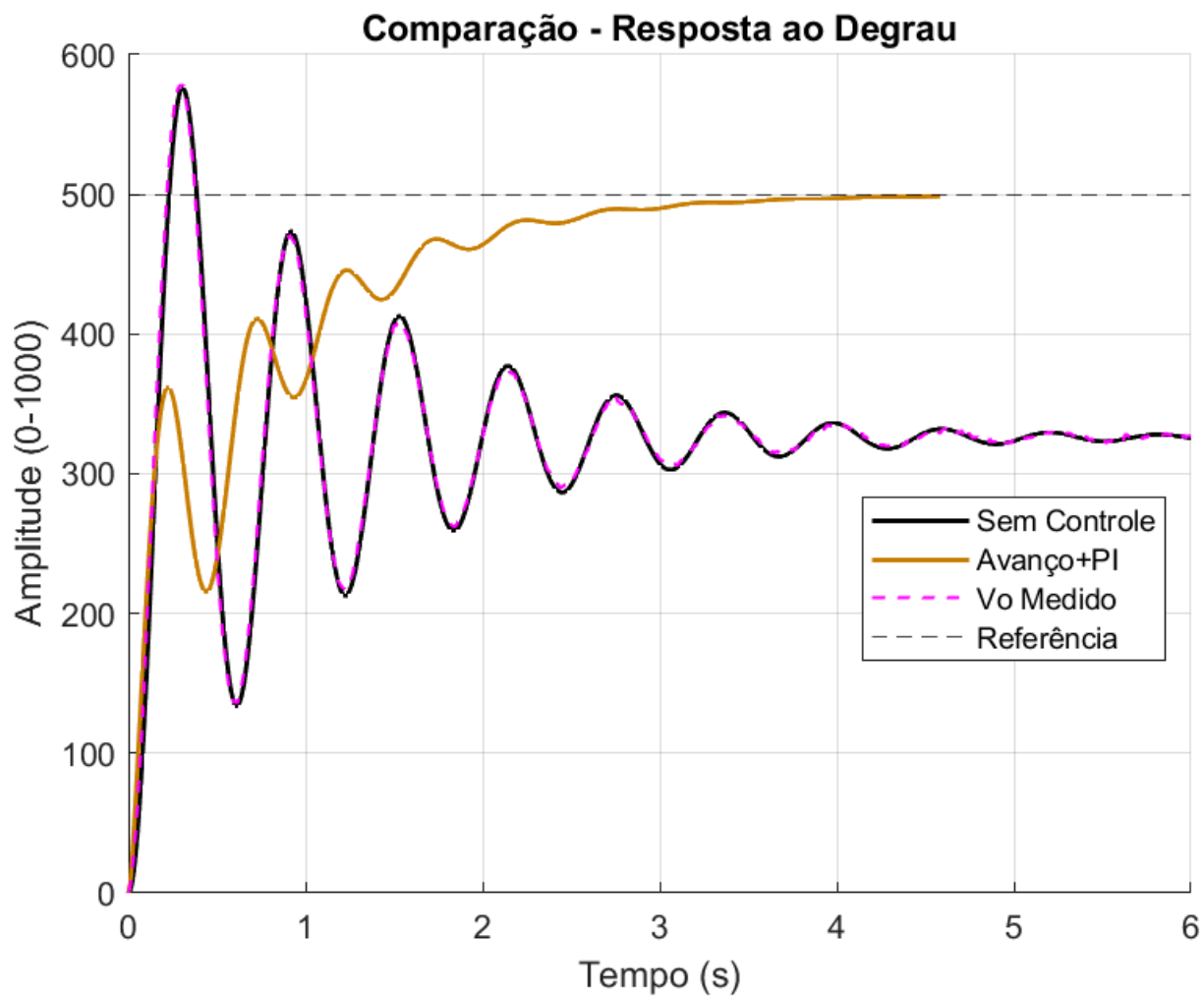
Para essa junção dos controladores, buscou combinar as características dos dois compensadores. Para isso, foram testadas várias configurações, e a escolhida foi:

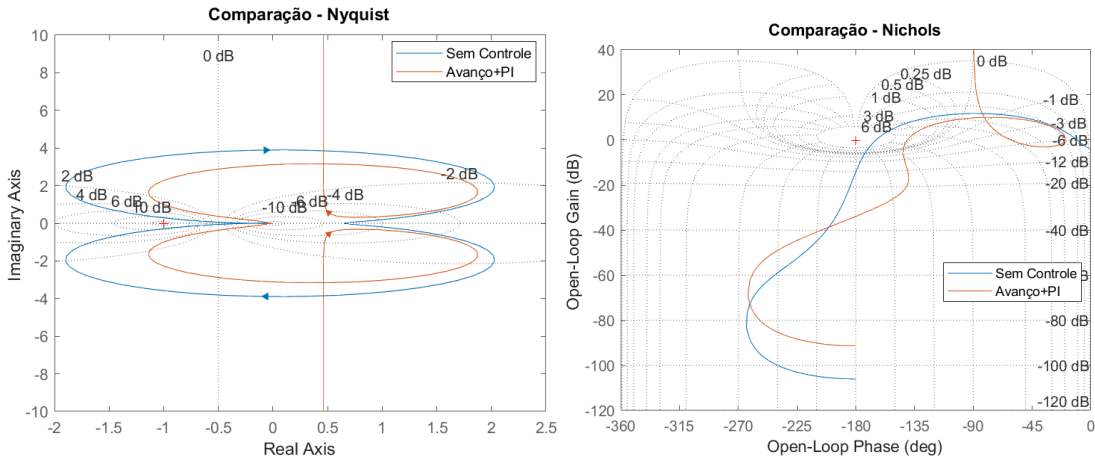
matlab:

zc_av2 = 0.90; pc_av2 = 0.30; Kc_av2 = 2; % Avanço

zc_pi2 = 0.95; Kc_pi2 = 1.90; % PI

Essa configuração apresentou 0% de sobressinal, tempo de estabilização de 4 s e erro nulo em regime permanente, resultando em melhoria em todos os aspectos comparado ao sistema original.





Para a implementação no Arduino, foi necessário juntar os dois controladores e obter a equação de diferenças resultante.

Parâmetros:

$z_{c_av2} = 0.90$; $pc_av2 = 0.30$; $K_{c_av2} = 2$;

$z_{c_pi2} = 0.95$; $K_{c_pi2} = 1.90$;

Termos intermediários:

Avanço:

$n0 = K_{c_av2} = 2$

$n1 = -K_{c_av2} \cdot z_{c_av2} = -2 \times 0.90 = -1.8$

$d0 = 1$

$d1 = -pc_av2 = -0.30$

PI:

$m0 = K_{c_pi2} = 1.9$

$m1 = -K_{c_pi2} \cdot z_{c_pi2} = -1.9 \times 0.95 = -1.805$

$e0 = 1$

$e1 = -1$

Coefficientes:

$b0 = n0 \cdot m0 = 2 \times 1.9 = 3.800$

$b1 = n0 \cdot m1 + n1 \cdot m0 = 2 \times (-1.805) + (-1.8) \times 1.9 = -7.030$

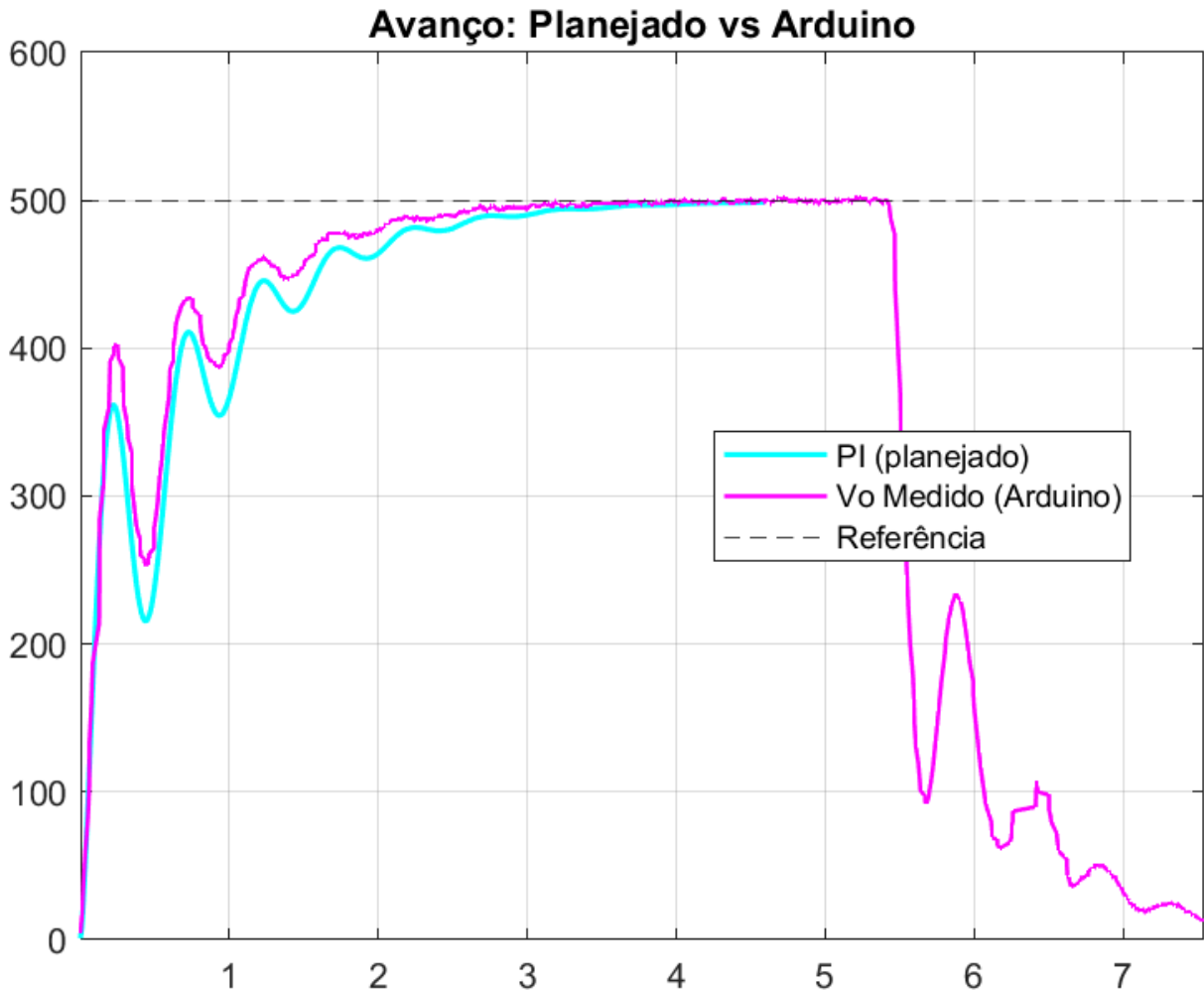
$b2 = n1 \cdot m1 = (-1.8) \times (-1.805) = 3.249$

$a1 = d0 \cdot e1 + d1 \cdot e0 = 1 \times (-1) + (-0.3) \times 1 = -1.300$

$a2 = d1 \cdot e1 = (-0.3) \times (-1) = 0.300$

Equação de diferenças:

$u[k] = b0 \cdot e[k] + b1 \cdot e[k-1] + b2 \cdot e[k-2] - a1 \cdot u[k-1] - a2 \cdot u[k-2]$



7. Controlador P I

Para o PID, a abordagem foi diferente: em vez de escolher o polo e os zeros e depois o ganho, optou-se por definir diretamente os ganhos.

```
Kp_pid = 0.25; Ki_pid = 2.25; Kd_pid = 0.025;
```

```
Cp = tf(Kp_pid, 1, T);
```

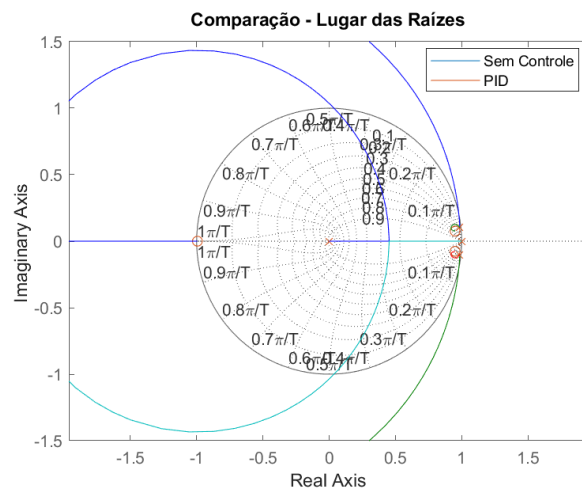
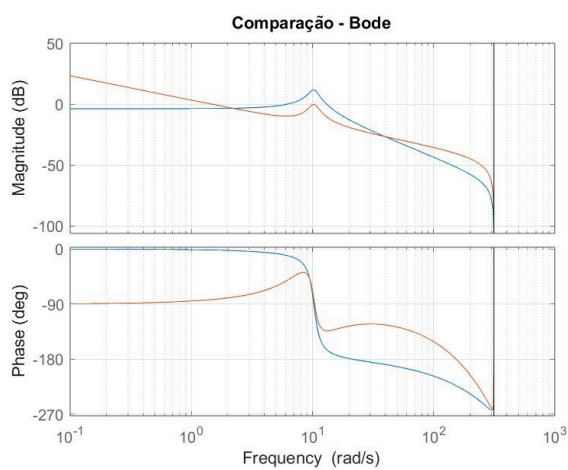
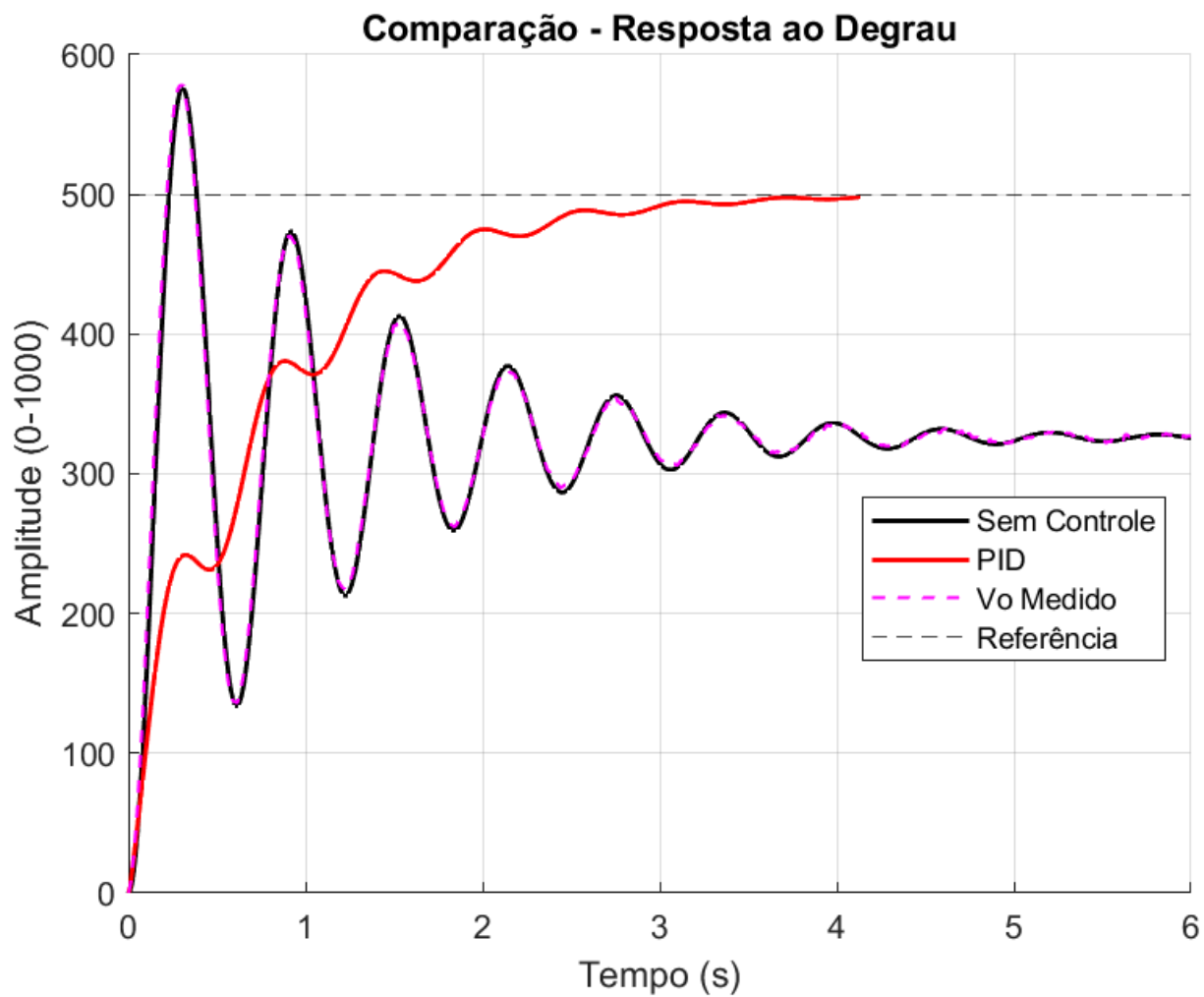
```
Ci = tf(Ki_pid*T, [1 -1], T);
```

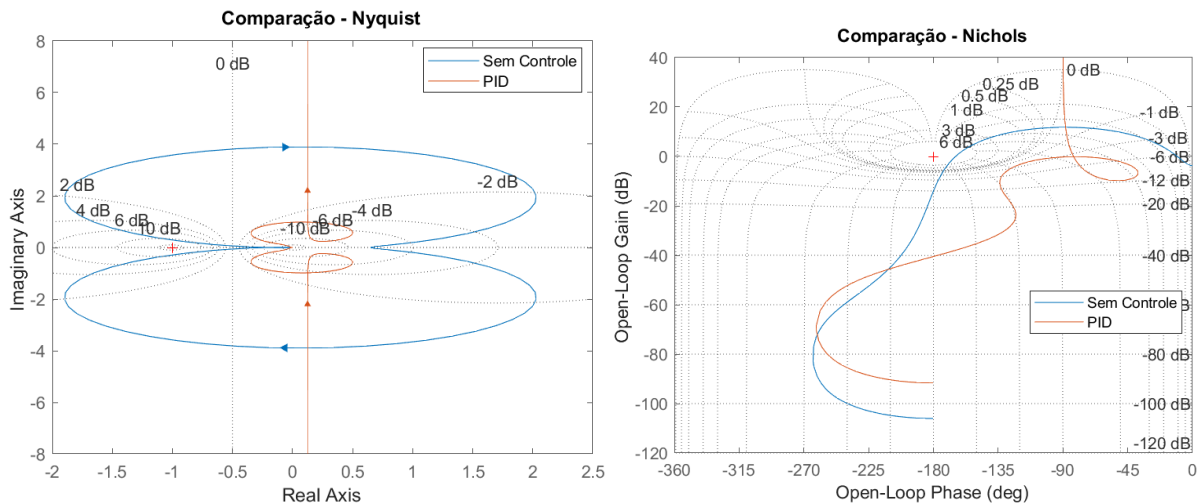
```
Cd = tf(Kd_pid/T*[1 -1], [1 0], T);
```

```
C_PID = Cp + Ci + Cd;
```

```
MF_PID = feedback(C_PID*G, 1);
```

Esse pequeno código foi o utilizado no MATLAB, ficando bem mais fácil de implementar no Arduino, pois os ganhos Kp, Ki e Kd são transferidos diretamente para o código, sem necessidade de conversão.

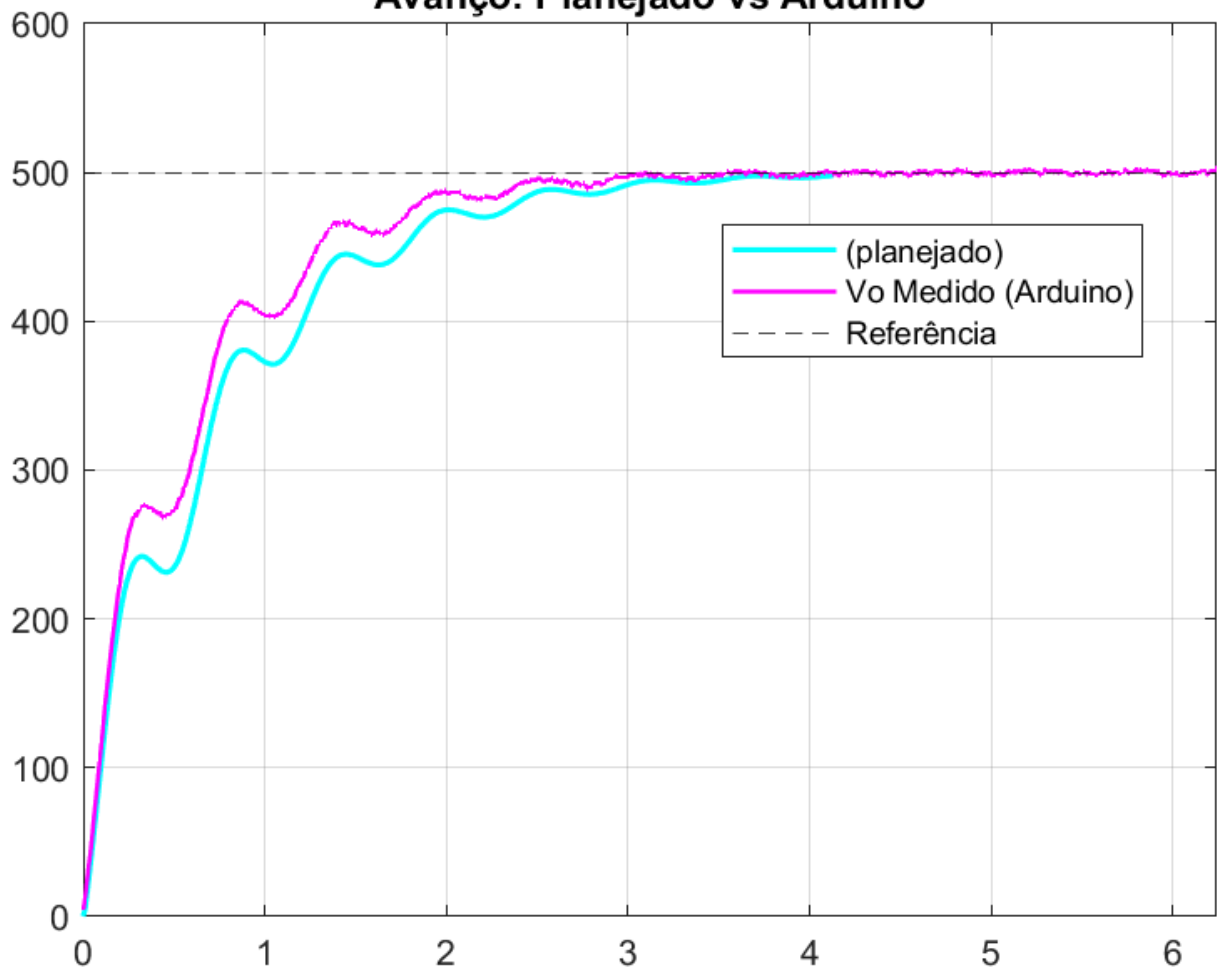




A conclusão é que o PID apresenta 0% de sobressinal, erro nulo em regime permanente, tempo de acomodação de 3 s e pouca oscilação durante o processo em comparação aos outros controladores.

Como esse controlador já foi pensado para a implementação, bastou inserir os ganhos no Arduino para obter esse resultado, que condiz com o que foi planejado.

Avanço: Planejado vs Arduino



8. Conclusao

Controlador	RiseT (s)	SettleT (s)	OS (%)	GanhoDC	Erro (%)	MG (dB)	MF (°)
Sem Controle	0.11	4.35	76.7	0.652	34.8	14.01	15.36
Proporcional	0.07	6.51	87.72	0.499	50.06	10.31	9.52
Avanço	0.01	0.16	26.89	0.888	11.2	11.67	58.68
PI	1.91	5.34	0.86	1.000	≈ 0	6.17	96.05
Avanço+PI	1.56	3.00	0	1.000	≈ 0	33.87	39.34
PID	1.72	2.96	0	1.000	≈ 0	40.5	97.32

As métricas avaliadas são: RiseT (tempo de subida), o tempo que a saída leva para sair de uma fração baixa até próximo do valor final; SettleT (tempo de acomodação), o tempo até a saída estabilizar dentro de uma faixa (tipicamente $\pm 2\%$) em torno do valor final; OS (sobressinal), o quanto a resposta ultrapassa o valor final, em porcentagem; GanhoDC, o ganho em regime permanente, onde 1 indica rastreamento perfeito; Erro (%), o erro de regime ao degrau, calculado como $(1 - \text{GanhoDC})$; MG (margem de ganho, em dB) e MF (margem de fase, em graus), que medem a robustez do sistema quanto maiores, mais distante da instabilidade.

Analisando os controladores, o sistema sem controle serve de referência, com sobressinal alto (76.7%), erro de regime de 34.8% e baixas margens, evidenciando um comportamento oscilatório e impreciso; o Proporcional não resolve, pois piora o sobressinal (87.7%) e apresenta o maior erro de regime (50%), confirmando que apenas o ganho é insuficiente; o Avanço é o mais rápido a estabilizar (0.16 s) e melhora a margem de fase (58.7°), mas, por não ter ação integral, mantém erro de regime (11.2%); o PI elimina o erro de regime ($\text{GanhoDC} = 1$) e oferece elevada margem de fase (96°) com sobressinal mínimo (0.86%), porém é lento e ainda oscilatório; o Avanço+PI combina o transitório rápido do avanço com o erro zero do PI, resultando em sobressinal nulo, acomodação em 3 s e boa margem de ganho (33.9 dB); e o PID apresenta o melhor desempenho global, reunindo sobressinal nulo, a menor acomodação (2.96 s), erro zero e as maiores margens de estabilidade (MG de 40.5 dB e MF de 97°), graças à ação derivativa que garante o amortecimento ausente nos demais.

9. Referências

AGUIRRE, Luis Antonio. Controle de Sistemas Amostrados. [S. l.]: [s. n.], 2020.

[PROFESSOR]. Material da disciplina Controle II. Disponível em: <https://oshwlab.com/daniel_bezerra/sallen-key-duino>. Acesso em: 28 jun. 2026.

[SEU NOME]. Sallen-Key-Duino: códigos do projeto de Controle II. GitHub, 2026. Disponível em: <<https://github.com/luanjjbr/Sallen-Key>>. Acesso em: 28 jun. 2026.